

CAPÍTULO 1

RESOLUCIÓN DE PROBLEMAS

1 Introducción

Informática → ciencia que estudia el análisis y resolución de problemas utilizando computus..
→ se vincula especialmente con la Matemática (= ciencia)

Ciencia → relacionada con una metodología fundamentada y racional para el estudio y resolución de los problemas.

definición “**problema**”

- Cuestión o proposición dudosa, que se trata de aclarar o resolver.
- Enunciado encaminado a averiguar el modo de obtener un resultado cuando se conocen ciertos datos.

La **resolución de problemas mediante una computadora** consiste en dar una adecuada formulación de pasos precisos a seguir.

Cuando una persona indica a otra como resolver un problema, se utiliza un lenguaje común y corriente para explicar. Es difícil transmitir el mensaje y con frecuencia se malinterpretan las instrucciones y se ejecuta incorrectamente la solución obteniéndose errores.

Con una computadora, **no** se utilizan indicaciones **ambiguas**. Ante cada orden resulta fundamental tener una única interpretación de lo que hay que realizar. Una máquina no posee la capacidad de decisión del ser humano para resolver situaciones no previstas. Si al dar una orden a la computadora se produce una situación no contemplada, será necesario abortar esa tarea.

Además, para poder indicar a la computadora las órdenes que debe realizar es necesario previamente entender exactamente lo que se quiere hacer. Es fundamental conocer con qué información se cuenta y qué tipo de transformación se quiere hacer sobre ella.

2 Etapas en la resolución de problemas con computadora

La resolución de problemas utilizando como herramienta una computadora (**no se resume únicamente en la escritura de un programa**) abarca todos los aspectos que van desde interpretar las necesidades del usuario hasta verificar que la respuesta brindada es correcta.

Las etapas son:

1) **Análisis del problema:** se analiza el problema en su contexto del mundo real. Deben obtenerse los requerimientos del usuario. El resultado de este análisis es un modelo preciso del ambiente del problema y del objetivo a resolver. Dos componentes importantes de este modelo son los datos a utilizar y las transformaciones de los mismos que llevan al objetivo.

2) **Diseño De una Solución:** Una técnica de diseño en la resolución de problemas consiste en la identificación de las partes (subproblemas) que componen el problema y la manera en que se relacionan. Cada subproblema debe tener un objetivo específico, debe resolver una parte del problema original. La integración de las soluciones de los subproblemas es lo que permitirá obtener la solución buscada.

3) **Especificación de algoritmos:** La solución de cada subproblema debe ser especificada a través de un algoritmo. Esta etapa busca obtener la secuencia de pasos a seguir para resolver el problema. La elección del algoritmo adecuado es fundamental para garantizar la eficiencia de la solución.

4) **Escritura de programas:** Un algoritmo es una especificación simbólica que debe

convertirse en un programa real sobre un lenguaje de programación concreto. Un programa escrito en un lenguaje de programación determinado es traducido automáticamente al lenguaje de máquina de la computadora que lo va a ejecutar. Esta traducción, denominada compilación, permite detectar y corregir los errores sintácticos que se cometen en la escritura del programa.

5) **Verificación**: Una vez que tiene un programa escrito en un lenguaje de programación se debe verificar que su ejecución produce el resultado deseado.

A continuación se trabajará sobre el concepto de Algoritmo como forma de especificación de soluciones concretas para la resolución de problemas con computadora.

3 Algoritmo

Algoritmo → Es una secuencia ordenada de pasos elementales, exenta de ambigüedades, que lleva a la solución de un problema dado en un tiempo finito

Paso elemental → aquel que no puede volver a ser dividido en otros más simples.

instrucción = sinónimo de paso elemental.

Hay aspectos se descartan, se suponen. Hay aspectos que no pueden obviarse o suponerse porque el algoritmo perdería lógica.

Un desarrollador de algoritmos deberá reconocer los aspectos importantes y tratar de simplificar su especificación de manera de seguir resolviendo el problema con la menor cantidad de órdenes posibles.

4 Pre y Postcondiciones de un algoritmo

Precondición → info que se conoce como verdadera antes de comenzar el algoritmo.

- **Supone que se dispone de todos los elementos para llevar a cabo la tarea**

Postcondición → info que se conoce como verdadera al concluir el algoritmo si se cumple adecuadamente el requerimiento pedido.

5 Elementos que componen un algoritmo

1 Secuencia de Acciones

- **Secuencia de Acciones**: está formada por una serie de instrucciones que se ejecutan una a continuación de la otra.

- **Flujo de control** del algoritmo: orden en el cual deben ejecutarse los pasos individuales.

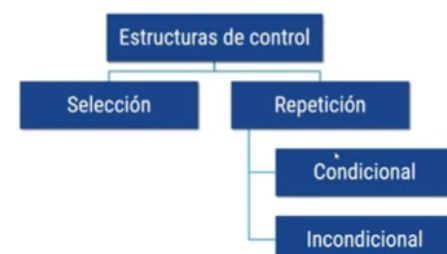
- flujo de control secuencial: ejecución uno a uno de los pasos, desde el 1ro hasta el último.

- **Estructuras de control**: son construcciones algorítmicas que permiten cambiar el flujo de control secuencial del algoritmo.

Con ellas es posible seleccionar un determinado sentido de acción entre un par de alternativas específicas o repetir automáticamente un grupo de instrucciones.

2 Selección

La **escritura de soluciones** a través de una **secuencia de órdenes** implica que siempre se ejecuten los mismos pasos, sin considerar que pueden existir opciones alternativas de acción para distintos casos. Hay que definir a priori cuál de las diferentes alternativas se presentarán al momento de la resolución del problema. Cuando



existen alternativas diferentes es imposible contar con cuál de ellas se trabajará, antes de comenzar la ejecución de la secuencia de acciones.

A través de la **selección** se incorpora, a la especificación del algoritmo, la capacidad de decisión. Así será posible seleccionar una de dos alternativas de acción posibles durante la ejecución del algoritmo. La **selección** permite que en un algoritmo se puedan considerar alternativas.

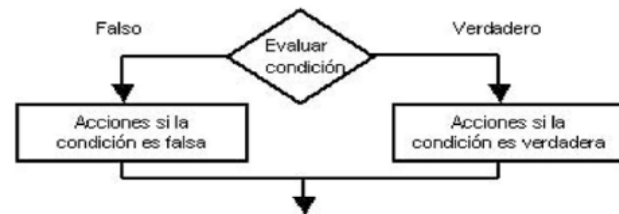


Figura 1.2: Estructura Si-Entonces-Sino

La selección se notará de la siguiente forma:

si (condición) → acción a realizar si la condición es verdadera

sino → acción acciones a realizar si la condición es falsa

Condición es una expresión que al ser evaluada puede tomar uno de dos valores posibles: **verdadero** o **falso**.

Si la condición resulta verdadera, solo se ejecutarán las acciones de (1)

En caso contrario, si la condición resulta ser falsa, solo se ejecutarán las acciones de (2).

Puede suceder que no existan acciones a realizar si la condición es falsa. Entonces, la estructura de control es **Si-Entonces**.



Figura 1.3: Estructura Si-Entonces

Precondiciones: Estoy parado frente a la heladería y hay helado de frutilla.

1. Entrar a la heladería
2. Pedir un helado de frutilla
3. Pagar
4. Salir de la heladería

Es una precondición estricta. ¿Qué pasaría si no se puede suponer?

Precondiciones: Estoy parado frente a la heladería.

1. Entrar a la heladería
2. Si hay helado de frutilla
3. Pedir un helado de frutilla
4. Sino
5. Pedir helado de chocolate
6. Pagar
7. Salir de la heladería

3 Repetición

La computadora puede ejecutar, de manera repetitiva, algunos pasos elementales de un algoritmo. Esto puede considerarse una extensión natural de la secuencia.

Repetición: estructura de control que permite al algoritmo ejecutar un conjunto de instrucciones un número de veces fijo y conocido de antemano.

La **notación** a utilizar es: **repetir (N)** (acción o acciones a realizar N veces)

4 Iteración

Sucede cuando **se desconoce el número de veces** que deben repetirse las acciones. Entonces, además de una serie de pasos elementales que se repiten; es necesario un mecanismo que lo detenga.

iteración: estructura de control que permite al algoritmo ejecutar en forma repetitiva un conjunto de acciones utilizando una condición para indicar su finalización.

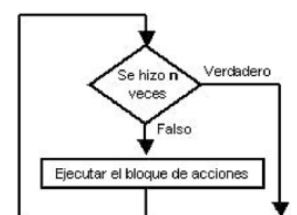


Figura 1.4: Estructura repetitiva

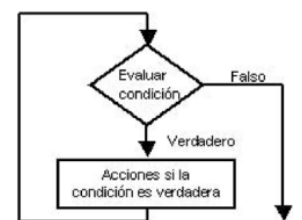


Figura 1.5: Estructura iterativa

La notación es: **mientras** (condición) → acciones a realizar si la condición es verdadera.

Las acciones serán ejecutadas si la condición es verdadera. La **primera vez**, antes de ejecutar alguna de las acciones de la iteración, lo primero que se realiza es la **evaluación de la condición**. Luego de comprobar que es verdadera se procede a ejecutar el conjunto de acciones.

Si la condición resultara falsa, el contenido del mientras no se ejecutará.

Es importante que las acciones de la iteración modifiquen el valor de verdad de la condición a fin de garantizar que la iteración termine en algún momento.

6 Importancia de la indentación en las estructuras de control

Las instrucciones que pertenecen a una estructura de control deben tener una sangría mayor que la utilizada para escribir el comienzo de la estructura. Para identificar el comienzo y fin del conjunto de instrucciones. A esta sangría se la denomina **indentación**. Este concepto se aplica a las estructuras de control vistas previamente: *selec, repet, iterac.*

7 Conclusión

El uso de algoritmos permite expresar, de forma clara, la manera en que un problema debe ser resuelto.

EJERCITACIÓN → págs 25 - 27

CAPÍTULO 2

ALGORITMOS Y LÓGICA INTRODUCCIÓN AL LENGUAJE DEL ROBOT

1 Lenguajes de Expresión de Problemas. Tipos de Lenguajes. Sintaxis y semántica en un Lenguaje.

Lenguaje natural → varios significados para una palabra = ambiguo y
→ admite varias combinaciones para armar un enunciado.

Estas son condiciones “indeseables” para un lenguaje en Informática.

El significado de cada instrucción del lenguaje debe ser exactamente conocido y no se pueden admitir diferentes interpretaciones.

Un **lenguaje de expresión de problemas** contiene un conjunto finito y preciso de instrucciones o primitivas utilizables para especificar la solución buscada.

Un **lenguaje de expresión de problemas** debe reunir las siguientes **características**:

- estar formado por un número de instrucciones finito.
- ser completo, todas las acciones de interés deben poder expresarse con dicho conjunto de instrucciones.
- Cada instrucción debe tener un significado (efecto) preciso.
- Cada instrucción debe escribirse de modo único.

1 Tipos de Lenguajes

No siempre los problemas se expresan con primitivas que representan un subconjunto preciso del lenguaje natural: se puede utilizar un sistema de símbolos gráficos, puede

emplearse una simbología matemática, puede crearse un lenguaje especial orientado a una aplicación, pueden combinarse gráficas con texto, etc.

Cualquiera sea la forma del lenguaje elegido siempre respetará las características

2 Sintaxis y Semántica en un Lenguaje

La forma en que se debe escribir cada instrucción de un lenguaje y las reglas generales de expresión de un problema completo en un lenguaje constituyen su **sintaxis**.

Por ej: hay lenguajes que en su sintaxis tienen reglas tales como:

- Indicar el comienzo y fin del algoritmo con palabras especiales.
- Indicar el fin de cada instrucción con un separador (ej punto y coma).
- Encerrar, entre palabras clave, bloques de acciones comunes a una situación del problema (ej todo lo que hay que hacer cuando la condición es verdadera dentro de la estructura de control de selección).
- Indentar adecuadamente las instrucciones.

El significado de cada instrucción del lenguaje y el significado global de determinados símbolos del lenguaje constituyen su **semántica**.

Como cada instrucción debe tener un significado preciso, la semántica de cada instrucción es una indicación exacta del efecto de dicha instrucción.

2 Ambiente de programación del robot (Rinfo).

El **robot Rinfo** que se utiliza posee las siguientes capacidades básicas:

1. Se mueve.
2. Se orienta hacia la derecha = gira 90 grados en el sentido de las agujas del reloj.
3. Tiene sensores visuales = puede reconocer dos formas de objetos preestablecidas: flores y papeles.
4. Lleva una bolsa donde puede transportar flores y papeles. puede recoger y depositar estos 2 objetos en una esquina, de a uno a la vez. La bolsa posee capacidad ilimitada.
5. Puede realizar cálculos simples.
6. Puede informar los resultados obtenidos.

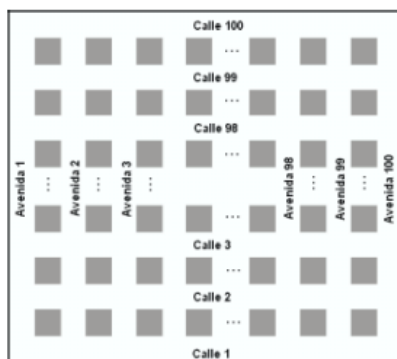


Figura 2.1: La ciudad del robot

La ciudad está formada por calles (100) y avenidas (100).

avenidas = arterias verticales

calles = arterias horizontales

esquinas = intersección de una avenida y una calle.

arterias = rectas // **esquina** = punto de intersección entre dichas rectas.

- La **esquina** se representará por dos coordenadas: la primera indicará el **número de avenida** y la segunda el **número de calle**.

- Las flores y los papeles se encuentran siempre en las esquinas. Pueden haber varias flores y papeles en cada esquina.

- La ciudad es de 100 calles y 100 avenidas;

Sintaxis	Semántica
Iniciar(robot,posición)	Instrucción primitiva que posiciona al robot en la esquina indicada orientado hacia el norte. <i>En este curso, siempre debemos posicionar al robot en la esquina (1,1) para comenzar su ejecución.</i>
derecha	Instrucción primitiva que cambia la orientación del robot en 90° en sentido horario respecto de la orientación actual.
mover	Instrucción primitiva que conduce al robot de la esquina en la que se encuentra a la siguiente, respetando la dirección en la que está orientado. Es responsabilidad del programador que esta instrucción sea ejecutada dentro de los límites de la ciudad. En caso contrario se producirá un error y el programa será abortado.
tomarFlor	Instrucción primitiva que le permite al robot recoger una flor de la esquina en la que se encuentra y ponerla en su bolsa. Es responsabilidad del programador que esta instrucción sea ejecutada solo cuando haya al menos una flor en dicha esquina. En caso contrario se producirá un error y el programa será abortado.
tomarPapel	Instrucción primitiva que le permite al robot recoger un papel de la esquina en la que se encuentra y ponerlo en su bolsa. Es responsabilidad del programador que esta instrucción sea ejecutada solo cuando haya al menos un papel en dicha esquina. En caso contrario se producirá un error y el programa será abortado.

- El andar del robot queda asociado con un paso que equivale a una cuadra de recorrido;
- Se reducen los datos en el modelo para tratar sólo con flores y papeles;
- Se aceptan convenciones (el robot solo inicia sus recorridos en la posición (1,1) de la ciudad);
- El robot ve y reconoce las flores y los papeles.

1 Operaciones en el ambiente del robot Rinfo

(La sintaxis en este lenguaje es sensible a mayúsculas y minúsculas)

depositarFlor	Instrucción primitiva que le permite al robot depositar una flor de su bolsa en la esquina en la que se encuentra. Es responsabilidad del programador que esta instrucción sea ejecutada solo cuando haya al menos una flor en dicha bolsa. En caso contrario se producirá un error y el programa será abortado.
depositarPapel	Instrucción primitiva que le permite al robot depositar un papel de su bolsa en la esquina en la que se encuentra. Es responsabilidad del programador que esta instrucción sea ejecutada solo cuando haya al menos un papel en dicha bolsa. En caso contrario se producirá un error y el programa será abortado.
PosAv	Identificador que representa el número de avenida en la que el robot está actualmente posicionado. Su valor es un número entero en el rango 1..100 y no puede ser modificado por el programador.
PosCa	Identificador que representa el número de calle en la que el robot está actualmente posicionado. Su valor es un número entero en el rango 1..100 y no puede ser modificado por el programador.
HayFlorEnLaEsquina	Proposición atómica cuyo valor es V si hay al menos una flor en la esquina en la que el robot está actualmente posicionado, ó F en caso contrario. Su valor no puede ser modificado por el programador.
HayPapelEnLaEsquina	Proposición atómica cuyo valor es V si hay al menos un papel en la esquina en la que el robot está actualmente posicionado, ó F en caso contrario. Su valor no puede ser modificado por el programador.
HayFlorEnLaBolsa	Proposición atómica cuyo valor es V si hay al menos una flor en la bolsa del robot, ó F en caso contrario. Su valor no puede ser modificado por el programador.
HayPapelEnLaBolsa	Proposición atómica cuyo valor es V si hay al menos un papel en la bolsa del robot, ó F en caso contrario. Su valor no puede ser modificado por el programador.
Pos(Avenida, Calle)	Instrucción que requiere dos valores Avenida y Calle, cada uno de ellos en el rango 1..100, y posiciona al robot en la esquina determinada por el par (Avenida,Calle) sin modificar la orientación del robot.
Informar	Instrucción que permite visualizar en pantalla el contenido almacenado en alguna variable o valores literales.

2 Estructura general de un programa

Comienza con la palabra clave **programa**, seguida por un identificador que determina el nombre del programa.

El cuerpo del programa principal es una secuencia de **sentencias**, delimitada por las palabras claves **comenzar** y **fin**.

En el programa se debe realizar un conjunto de declaraciones antes de comenzar el código

1ro se dispondrá de un sector para declarar el **área** que se utilizará (ciudad de 100 avenidas y 100 calles).

2do se declaran los $\neq$ tipos de **robot** que se utilizarán y el conjunto de instrucciones que cada robot debe ejecutar

3ro Habrá otro espacio asignado para los **módulos**.

4to se indicarán las **variables** que se asocian a cada robot declarado.

5to entre las palabras **comenzar** y **fin** se escribe el **código** que indica en cuáles áreas se pueden mover los robots declarados y una instrucción que indica al robot que comience a ejecutar las órdenes definidas.

programa *nombre_del_programa*

areas

Se declara una única área que comprende toda la ciudad

robots

Se declara un único tipo de robot "robot1" cuyo código corresponde a la solución del problema que se está resolviendo

variables

Se declara una variable que representa al robot, será llamada Rinfo

comenzar

*Se asigna el área donde se desplazará Rinfo (en este curso toda la ciudad).
Se indica el inicio para que cada robot se ejecute (en este curso solo se indica el comienzo de ejecución de Rinfo).*

fin

Comentarios lógicos

- Los problemas para poder ser resueltos, deben entenderse
- Las soluciones deben ser entendidas por otros.
- Para que las soluciones sean claras y legibles, se deben agregar comentarios aclaratorios.
- un algoritmo adecuadamente escrito debería interpretarse sólo leyendo los comentarios.
- expresar en forma sintética la función de una instrucción
- los comentarios se intercalan en el algoritmo con algún símbolo inicial que indique que se trata de un comentario (Rinfo → llaves)

3 Estilo de programación

objetivo: mejorar la legibilidad del código para que sea sencillo entenderlo, modificarlo, adaptarlo y usarlo, maximizando la productividad y minimizando el costo de desarrollo y mantenimiento.

Al escribir un programa se deben respetar las siguientes **reglas de indentación**:

- La palabra clave **programa** debe comenzar en la **1er columna**.
- Las palabras claves **comenzar** y **fin** deben comenzar en la **1er columna**.
- Las sentencias del cuerpo del programa deben comenzar **dos columnas** más a la derecha

- Las **sentencias** que pertenecen al **cuerpo** de una **estructura de control** deben comenzar **dos columnas** más a la derecha que la palabra clave que identifica a la estructura de control.
- La **indentación** es la única forma de indicar si una sentencia pertenece o no a la estructura de control.

Todas las **palabras claves** deben ser escritas con **minúsculas**, si su nombre está compuesto por + de una p, de la segunda palabra en adelante, comienzan con mayúscula. Las **variables y los procesos** del sistema deben comenzar cada palabra que compone su nombre con mayúscula y las demás minúsculas.

4 Ambiente de programación

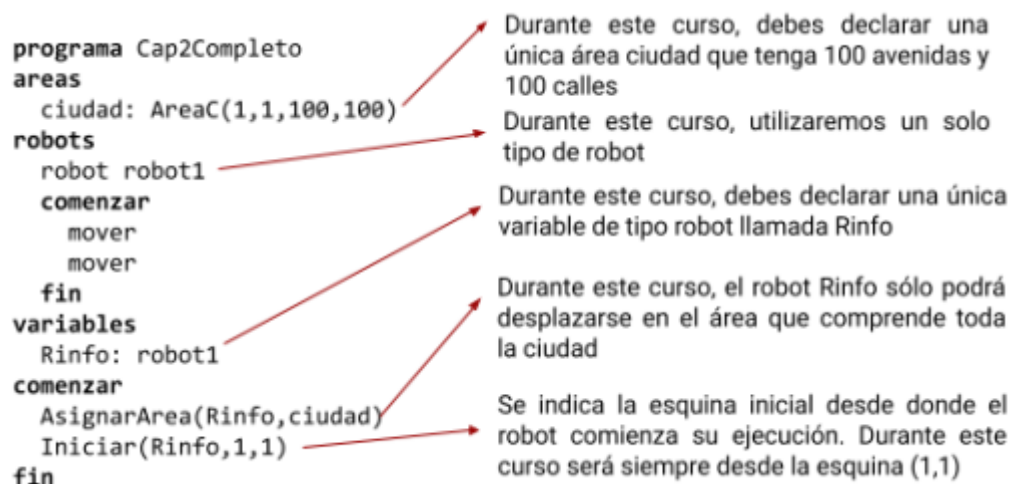
Ambiente de programación = herramienta que permite cubrir las distintas etapas en el desarrollo de un programa (desde la codificación del algoritmo en un lenguaje de programación hasta su ejecución) a fin de obtener los resultados esperados.

Cada ambiente de programación trabaja sobre un lenguaje específico.

Para codificar un algoritmo en el lenguaje del robot Rinfo es necesario realizar las siguientes tres etapas:

1. **Escribir el programa:** se escriben los pasos a seguir usando la sintaxis descripta.
2. **Compilar el programa:** para lograr que la computadora ejecute el programa escrito hay que traducirlo a un lenguaje que la computadora comprenda. Esta etapa permite detectar los errores de sintaxis.
3. **Ejecutar el programa:** una vez compilado puede ejecutarse.

5 Comenzando a trabajar



3 Estructuras de Control

1 Secuencia

= conjunto de instrucciones que se ejecutarán una a continuación de otra.

No existe en el lenguaje del robot una instrucción para detenerlo. Ocurrirá al terminar el programa, con la palabra **fin**.

Informar(PosAv, PosCa) muestra los valores retornados por las instrucciones PosAv y PosCa.

2 Selección

permite al robot seleccionar una de dos alternativas posibles

Sintaxis: **si** (condición)

acción o bloque de acciones a realizar en caso de que la condición sea verdadera

sino

acción o bloque de acciones a realizar en caso de que la condición sea falsa

si no necesita realizar acciones cuando la condición es falsa, puede omitirse la palabra sino

si (condición)

acción o bloque de acciones a realizar en caso de que la condición sea verdadera

2 Repetición

Si se desea realizar una acción o acciones un número fijo de veces:

repetir N

acción o bloque de acciones a realizar

mientras (condición)

acción o bloque de acciones a realizar mientras la condición sea verdadera

4 Iteración

para visualizar el número de calle donde el robot quedó parado debe utilizarse **PosCa** ya que no es posible calcular el valor a priori

Análisis de la sintaxis del robot

Sintaxis del robot	
Iniciar	HayFlorEnLaEsquina
mover	HayPapelEnLaEsquina
derecha	HayFlorEnLaBolsa
tomarFlor	HayPapelEnLaBolsa
tomarPapel	PosAv
depositarFlor	PosCa
depositarPapel	
Pos	
Informar	

4 Proposiciones atómicas y moleculares, simbolización y tablas de verdad

En un problema real se requiere combinar expresiones para poder representar una situación a evaluar. La **selección** e **iteración**, requieren para su funcionamiento, la evaluación de una condición. Estas condiciones se conocen como **proposiciones**.

Proposición = expresión de la cual tiene sentido decir si es verdadera o falsa, o sea es posible asignarle un valor de verdad (verdadero o falso, no ambos).

1 Proposiciones atómicas y moleculares

En Lógica → **atómico** = “algo que no puede ser dividido nuevamente”

Una proposición es considerada atómica si no puede ser descompuesta en otras proposiciones.

Ejemplos

- La casa es roja.
- Hoy es lunes.
- He llegado al final del recorrido.
- Estoy ubicado a 3 metros de altura.

Cuando en una expresión se unen varias proposiciones atómicas se forma una proposición **molecular** o **compuesta**. La unión se realiza mediante **conectivos lógicos** ó **términos de enlace**.

Ejemplos

- **No** es cierto que la luna esté hecha de queso.
- La vaca está cansada **y no** dará leche.
- Hace calor **ó** hay mucha humedad.
- Hay papel en la bolsa **y** hay papel en la esquina.

Los términos de enlace son: “y”, “o”, “no”.

“y”, “o” → para conectar proposiciones atómicas;

“no” → solamente se coloca frente a una proposición atómica

Una proposición es atómica si no tiene conectivos lógicos, sinó es molecular.

2 Simbolización

Es importante simbolizar las proposiciones atómicas, moleculares y conectivos lógicos para facilitar las operaciones

Conectivo	Simbolización en ambiente de programación de Rinfo
y	&
o	
no	~

Se utilizarán letras minúsculas para simbolizar las proposiciones atómicas.

El **proceso para simbolizarlas** consiste en tres pasos:

1. Determinar cuáles son las proposiciones atómicas que la componen.
2. Simbolizar las proposiciones.
3. Unir las proposiciones con los conectivos.

Debe tenerse en cuenta si aparece una coma, la cual separa las dos primeras proposiciones de la tercera. Cuando se obtiene la simbolización debe respetarse ese orden. la palabra **PERO** actúa como el conectivo lógico “y”.

3 Tablas de verdad. Repaso

Proposición = expresión de la cual tiene sentido decir si es verdadera o falsa.

En la **tabla de verdad** de una **proposición molecular** se muestran todas las combinaciones de los valores de verdad de las proposiciones atómicas que la conforman.

1 Conjunción. Tabla de verdad

La **conjunción** de dos proposiciones es cierta si ambas proposiciones lo son.

En Lógica se pueden unir dos proposiciones para formar una conjunción. No se requiere que el contenido de una tenga relación con el de la otra.

La selección utiliza la conjunción de dos proposiciones que para ser verdadera necesitará que ambas proposiciones lo sean.

`si ((HayFlorEnLaBolsa) & ~(HayFlorEnLaEsquina))
depositarFlor`

p	q	p & q
V	V	V
V	F	F
F	V	F
F	F	F

2 Disyunción. Tabla de verdad

La disyunción entre dos proposiciones es cierta cuando al menos una de las proposiciones lo es. +2

La disyunción utiliza el término de enlace “o”. Basta con que una de las dos proposiciones sea verdadera para que la disyunción sea verdadera.

`si ((HayFlorEnLaEsquina) | (HayPapelEnLaEsquina))`

p	q	p q
V	V	V
V	F	V
F	V	V
F	F	F

3 Negación. Tabla de verdad

Dada una proposición p, su negación ~ p, permitirá obtener el valor de verdad opuesto. El valor de verdad de la negación de una proposición verdadera es falso y el valor de verdad de la negación de una proposición falsa es verdadero.

p	~ p
V	F
F	V

4 Utilización del paréntesis

`mientras ~(HayFlorEnLaEsquina)`

Hay proposiciones que tienen más de un término de enlace. En estos casos, uno de los términos de enlace es el más importante (término dominante), porque actúa sobre toda la proposición.

El operador “~” es el que tiene mayor prioridad, es el que primero se evalúa; seguido por “&” y “|”, estos poseen igual prioridad.

Ante una expresión que utilice varias conjunciones y/o disyunciones, se evaluarán de izquierda a derecha. Se puede alterar este orden natural de resolución, usando paréntesis.

Ejemplo

$\sim p / q$ es una disyunción entre $\sim p$ y q

$\sim (p | q)$ es la negación de una disyunción.

5 Teoremas de Lógica Proposicional

Es frecuente encontrar

- proposiciones que se necesita saber si son equivalentes entre sí.
- proposiciones que por su estructura no se comprenden fácilmente y requieren una simplificación o que sean expresadas de otra forma.

Para estos casos, están los teoremas fundamentales de Lógica Proposicional o Álgebra de Boole que ayudan para transformar una proposición en otra equivalente.

Teorema 1. El resultado de aplicar cualquiera de las tres operaciones (negación, conjunción o disyunción), a variables booleanas, es otra variable booleana y además el resultado es único.

Teorema 2. Ley de idempotencia. Tanto la suma como el producto de una variable booleana consigo misma da como resultado la misma variable: $p | p = p$ // $p \& p = p$

Teorema 3. Ley de involución. Una variable booleana negada dos veces, da como resultado la misma variable: $\sim (\sim p) = p$

Teorema 4. Ley conmutativa. Se define respecto a la conjunción y la disyunción, y nos dice que el orden de los sumandos (factores) no altera el resultado:

Respecto a la conjunción: $p \& q = q \& p$ // Respecto a la disyunción: $p | q = q | p$

Teorema 5. Ley asociativa. Se define:

Respecto a la conjunción: $p \& (q \& r) = (p \& q) \& r = p \& q \& r$

Respecto a la disyunción: $p | (q | r) = (p | q) | r = p | q | r$

Teorema 6. Ley distributiva.

Respecto a la conjunción: $p | (q \& r) = (p | q) \& (p | r)$

Respecto a la disyunción: $p \& (q | r) = p \& q | p \& r$

Teorema 7. Ley de absorción. $p | (p \& q) = p$ // $p \& (p | q) = p$

Teorema 8. Leyes De Morgan. Pueden ser generalizadas a "n" variables.

Respecto a la conjunción: $\sim (p \& q) = \sim p | \sim q$

Respecto a la disyunción: $\sim (p | q) = \sim p \& \sim q$

CAPÍTULO 3 DATOS

1 Conceptos de Control y Datos

Dato = elemento u objeto de la realidad que los algoritmos representan y son capaces de modificar y procesar.

Cuando se resuelven problemas con computadora, muchas veces se modelizan los objetos reales mediante objetos más abstractos, representables y entendibles sobre una computadora.

El mecanismo de resolución de problemas involucra un procesamiento de los datos, manejado por el control del algoritmo.

2 Representación de los Datos

Los algoritmos modifican objetos de la realidad. Su representación estará dada por sus características principales o por la info que nos interese conocer de ellos

Los datos a representar son aquellos de interés específico para resolver ese problema.

3 Variables

Además de poder moverse y de recoger o depositar flores y papeles, el robot puede manejar datos que le permiten representar ciertos atributos de los problemas que debe resolver. Por ej, calcular la cantidad de flores que lleva o recordar si en una esquina había más flores que papeles.

En general, durante la ejecución de un programa es necesario manipular información que puede cambiar continuamente. Por esto, se debe contar con un elemento que permita variar la información que se maneja en cada momento. Este elemento es la “**variable**”.

Variable → permite almacenar un valor modificable a lo largo de la ejecución del programa. Dicho valor representa un dato relevante para el problema a resolver.

1 Sintaxis para la declaración de variables

Se denominan **identificadores** a los nombres descriptivos que se asocian a los datos (variables) para representar, dentro del programa (código del robot), su valor.

En el ambiente lenguaje del robot Rinfo, existe un área donde se declaran las variables. En este ambiente no existe la posibilidad de declarar variables en el programa ([exceptuando la declaración del robot Rinfo](#)), sólo se permite declarar variables dentro del código del robot.

Para declarar una variable dentro del ambiente del robot Rinfo se debe elegir un nombre que la identifique y un tipo de datos al cual pertenece dicha variable.

La **sintaxis** para declarar una variable es:

```
robots
  robot robot1
  variables
    nombreVariable: tipoVariable
  comenzar
  ...
fin
```

→ Variable que puede ser utilizada mientras se ejecuta el código del robot.

Ahora quedan distinguidos dentro del robot **dos sectores**:

- un sector superior donde se declaran las variables de interés
- un sector inferior donde se detallan las instrucciones necesarias para que el robot resuelva el problema planteado.

4 Tipos de datos

Independientemente del lenguaje de programación, la definición de un tipo de dato implica considerar **tres aspectos fundamentales** que lo caracterizan:

- Identificar cuáles son los valores que puede tomar un dato.
- Definir cuáles son las operaciones que pueden aplicarse sobre los datos
- Indicar cuáles son las relaciones de orden que permiten compararlos.

1 Tipo de dato numérico (número)

Los elementos de este tipo de dato son los números enteros

Una computadora sólo puede representar un subconjunto finito de valores, con lo cual existirá un n° máx y un n° mín.

Las operaciones válidas para los números son: suma, resta, multiplicación y división entera. que se simbolizan como: +, -, *, /

Si el resultado de alguna operación sobrepasa el límite superior permitido para los elementos del tipo número, se dice que ocurre un overflow y si se encuentra por debajo del mínimo se dice que ocurre underflow.

Las relaciones de orden entre números son: igual, menor, mayor, menor o igual, mayor o igual y distinto.

Relación	Sintaxis	Ejemplo
Igualdad	=	A = B
Menor	<	3 * C < D
Mayor	>	C > 2 * B
Menor o igual	<=	A < (2 * C + 4)
Mayor o igual	>=	(B + C) > D
Distinto	<>	A <> B

Para declarar una variable numérica deberá utilizarse el sector de declaraciones dentro del robot.

“**número**” indica en el ambiente de programación del robot, que la variable será de tipo numérico, y esto implica que sus valores entrarán en el rango especificado anteriormente y que podrá utilizar cualquiera de las operaciones válidas para el tipo número.

El **operador de asignación** se representa con un dos puntos igual (:=) y permite dar valor a una variable

2 Tipo de dato lógico (boolean)

Este tipo de dato lógico puede tomar un valor **Verdadero** o **Falso**.

Si se utilizan variables de tipo booleano se puede asignar en ellas el resultado de cualquier expresión lógica o relacional.

variable **identicos** → guardará el valor V si el robot se encuentra ubicado en una esquina donde el valor de la avenida coincide con el valor de la calle y sino guardará F.

variable **esPositivo** → guardará el valor F si está ubicado sobre la calle 1 y sino guardará V.

```
programa numvariablesLogicas      comenzar
areas                             identicos:= (PosCa = PosAv)
  ciudad: AreaC(1,1,100,100)      esPositivo:= (PosCa > 1)
robots                             fin
  robot robot1                   variables
  variables                       Rinfo: robot1
    esPositivo: boolean          comenzar
    identicos: boolean           AsignarArea(Rinfo,ciudad)
                                Iniciar(Rinfo,1,1)
                                fin
```

5 Modificación de la información representada

Para guardar un valor determinado y posteriormente modificarlo.

Identificador := valor a guardar

El operador := se denomina operador de asignación y permite registrar o “guardar” el valor que aparece a derecha del símbolo en el nombre que aparece a izquierda del símbolo.

La instrucción **Informar**(cantidadCuadras) permitirá informar el último valor asignado a cantidadCuadras.

También, puede utilizarse el operador de asignación para modificar el valor de un identificador tomando como base su valor anterior

cantidadCuadras := cantidadCuadras + 1

La expresión de la derecha no es un valor constante sino que debe ser evaluado ANTES de realizar la asignación. El lado derecho indica que debe recuperarse el valor almacenado en

cantidadCuadras y luego incrementarlo en 1. El resultado de la suma será almacenado nuevamente en cantidadCuadras.

Ejercitación pág 83

CAPÍTULO 4 REPASO

1 Repaso de variables

.2 Repaso de expresiones lógicas

3 Ejemplos

Ejercitación